

Computer Vision Project – Exercise 5.1

Selective Search

Neelesh Babu, Simu Sulthana

28/06/2026

Implementation summary

The submitted code implements the complete selective search pipeline in `code/selective_search.py`. First, the image is over-segmented with the Felzenszwalb algorithm. Then each region is described using colour, texture, size and fill similarities. Neighbouring regions are merged in a hierarchical procedure, old similarities are removed, new similarities are calculated for the merged region, and the final region proposals are returned as bounding boxes. The script `code/main.py` runs the method on all nine images from `data/chrisarch`, `data/arthist` and `data/classarch`, and stores visualized results in the `results/` folder.

Questions

Q5.1.1 Felzenszwalb can already generate segmentations for the whole image, why do we need selective search at all?

Felzenszwalb gives one fixed over-segmentation of the image. This is useful, but object boundaries are not always equal to these initial small regions. Selective search is needed because it groups neighbouring segments step by step and therefore creates region proposals at different scales. In this way, small parts, medium parts and whole objects can all be proposed as possible object locations.

Q5.1.2 Proposal filtering is implemented in `main.py`. What are the criteria based on which the boxes are filtered and what might be the effect? Do you agree with the criteria? Can you think of additional ones?

The filtering removes duplicate rectangles, very small regions, almost full-image regions, and boxes with an extreme aspect ratio. The effect is that the output becomes cleaner and faster to inspect because many unlikely proposals are removed. However, this can also remove valid objects, for example very small objects or naturally elongated objects. I agree with the general idea of filtering, but the thresholds should depend on the dataset. Additional useful criteria could be a confidence score, minimum/maximum box area relative to the image size, non-maximum suppression to remove strongly overlapping boxes, or different aspect-ratio limits for different object categories.

Q5.1.3 Selective Search iteratively merges regions of arbitrary shapes. How do we obtain rectangles (box proposals) from that?

For every merged region, we compute the smallest axis-aligned bounding rectangle that contains all pixels of that region. This is done by storing `min_x`, `min_y`, `max_x` and `max_y`. The final box proposal is then `(min_x, min_y, width, height)`, where `width = max_x - min_x + 1` and `height = max_y - min_y + 1`.

Q5.1.4 Describe the effect of changing the k parameter of the Felzenszwalb algorithm on the final region proposals, and describe how changing the number of proposals affects the metric.

The `k/scale` parameter controls the coarseness of the initial Felzenszwalb segmentation. A smaller value produces more small initial regions, so selective search has more detailed parts to merge and usually produces more fine-grained proposals. A larger value produces fewer and larger initial regions, so the proposals are coarser and small objects may be missed. Increasing the number of proposals usually improves object recall and metrics such as MABO or mAP up to a point, because there is a higher chance that one proposal overlaps the true object well. After many proposals, the improvement becomes smaller, while runtime and false positives increase.

How to run

From the root folder:

```
pip install -r requirements.txt
python code/main.py
```

The default parameters resize large images to speed up the run. To run on full-size images, use:

```
python code/main.py --max-side 0
```